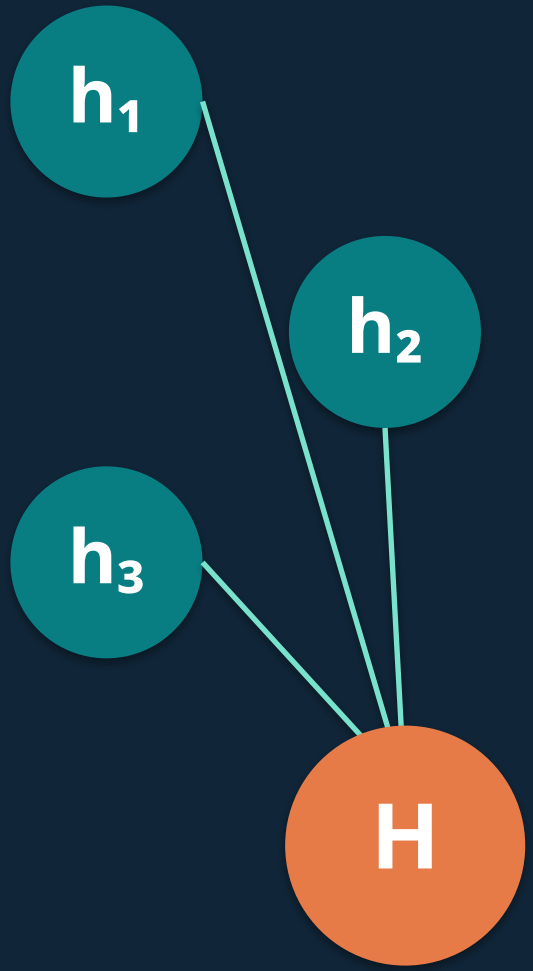
ENSEMBLE LEARNING

Từ cây quyết định đến mô hình tổ hợp

Đề 5 • Bagging và bỏ phiếu đa số

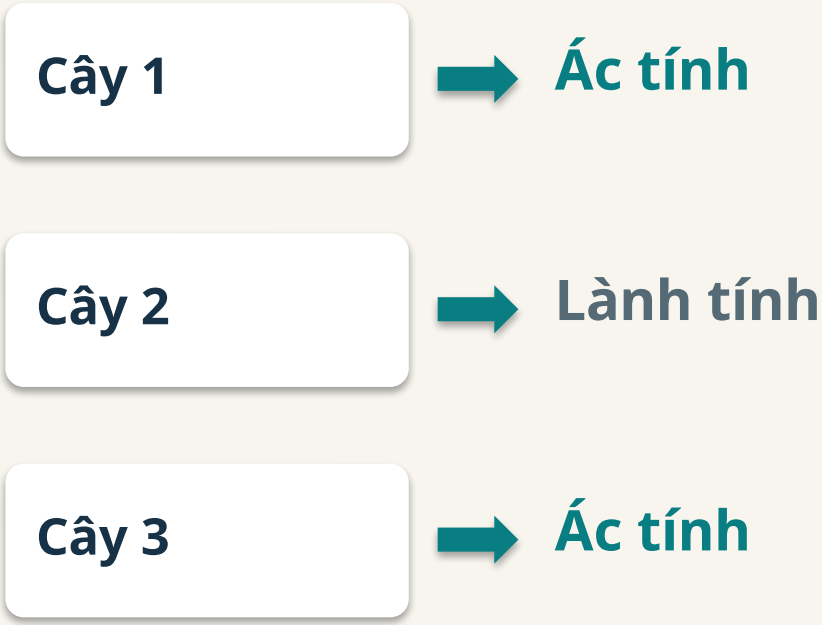
Nguyễn Nghĩa Hiếu • MSSV: 52400013
Trần Quốc Bảo • MSSV: 52400102
Giảng viên: TS. Trần Lương Quốc Đại



Nhiều mô hình chỉ hữu ích khi có sự đa dạng

Một mô hình có thể sai.
Một tập mô hình có thể bù lỗi cho nhau.

Điều kiện: mô hình cơ sở đủ tốt
và lỗi không hoàn toàn giống nhau.



Lộ trình: Nguyên lý → Ví dụ từng bước → Mã nguồn → Thực nghiệm

Bốn cách kết hợp thường gặp

BAGGING

Song song

Nhiều mẫu bootstrap
→ nhiều mô hình
→ tổng hợp dự đoán

Ví dụ: Bagged Trees

BOOSTING

Tuần tự

Mô hình sau tập trung
vào sai sót / phần dư
của mô hình trước

Ví dụ: AdaBoost, GBM

VOTING

Quy tắc kết hợp

Kết hợp nhãn dự đoán
hoặc xác suất lớp
của nhiều mô hình

Hard / Soft Voting

STACKING

Học cách kết hợp

Dự đoán ngoài fold
→ tạo đặc trưng mới
→ mô hình tầng hai

Meta-model

Vì sao Bagging có thể ổn định hơn?

MỘT CÂY DỄ DAO ĐỘNG

Đổi một phần dữ liệu train
→ ngưỡng chia có thể đổi
→ dự đoán có thể đổi.

Đó là trực giác về phương sai.

NHIỀU CÂY BÙ LỖI

Bootstrap tạo nhiều tập train.
Mỗi cây học hơi khác nhau.
Voting giảm ảnh hưởng của
một dự đoán riêng lẻ.

Cùng sai thì vẫn có thể sai.

Chìa khóa: cây đủ tốt + lỗi không quá giống nhau; thêm cây không bảo đảm tốt hơn.

Hard Voting và Soft Voting có thể cho kết quả khác

Quan sát x	Mô hình 1	Mô hình 2	Mô hình 3
P(ác tính | x)	0,51	0,51	0,01
Nhãn dự đoán	Ác tính	Ác tính	Lành tính

HARD → ÁC TÍNH

Đếm nhãn: 2 phiếu ác tính / 3 phiếu.

SOFT → LÀNH TÍNH

Trung bình P(ác tính) = 1,03 / 3 ≈ 0,343.

Bagging: lấy mẫu lại, học độc lập, rồi tổng hợp

01

Tập train

N quan sát
Giữ riêng test

02

Bootstrap

B mẫu; mỗi mẫu
N lượt có hoàn lại

03

Huấn luyện

Một cây / mẫu
Độc lập với nhau

04

Tổng hợp

Phiếu từng cây
→ lớp đa số

Seed 42: N = 426 mẫu train • B = 51 cây • max_depth = 3

Ví dụ bootstrap: 5 quan sát, 5 lượt lấy mẫu

Cùng 5 mẫu có x = [1, 2, 3, 4, 5]. Lấy mẫu theo chỉ số đếm từ 0.

Cây	Chỉ số bootstrap	Không có mặt
h1	[0, 0, 2, 4, 1]	3
h2	[0, 0, 1, 3, 4]	2
h3	[4, 2, 4, 1, 3]	0

CÓ LẶP

Cùng N lượt, nhưng không đủ N mẫu khác nhau.

P(một mẫu không được chọn) = $(1 - 1/N)^N \rightarrow e^{-1} \approx 36{,}8\%$ khi N lớn

Một cây học phép chia như thế nào?

Dữ liệu minh họa	0	1	2	3	4
Đặc trưng x	1	2	3	4	5
Nhãn	Lành	Lành	Ác	Ác	Ác

Bootstrap h_1: [0, 0, 2, 4, 1]
→ 3 lành tính, 2 ác tính

Gini trước chia
$= 1 - (3/5)^2 - (2/5)^2 = 0{,}48$

Chọn ngưỡng x ≤ 2,5

Trái: 3 lành → Gini = 0
Phải: 2 ác → Gini = 0
Gini sau chia = 0 | Giảm = 0,48

Cùng một mẫu mới, ba cây cho hai phía phiếu

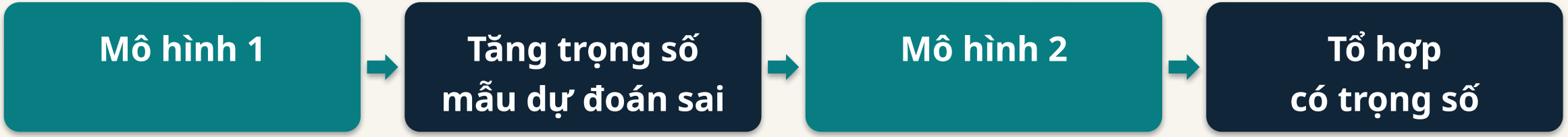
Mẫu mới x = 2,8. Mỗi cây: x ≤ ngưỡng → lành; x > ngưỡng → ác.

Cây	Ngưỡng học được	So sánh x = 2,8	Phiếu
h1	2,5	2,8 > 2,5	Ác tính (0)
h2	3	2,8 ≤ 3	Lành tính (1)
h3	2,5	2,8 > 2,5	Ác tính (0)

2 phiếu ác : 1 phiếu lành → chọn ác tính (0)

Cùng ví dụ slide 7–8 • Số liệu tính bởi examples/toy_bagging.py

Boosting: học tiếp từ những sai sót trước



Ví dụ AdaBoost nhị phân
Tăng chú ý vào mẫu bị dự đoán sai.

Mẫu sai: tăng trọng số.
Mẫu đúng: giảm trọng số tương đối.
Mô hình sau học lại trên các trọng số mới.

20% → 50%

Trong ví dụ, nhóm mẫu sai
đang mang 20% tổng trọng số;
sau cập nhật mang 50%.

Stacking học một mô hình để kết hợp các mô hình

TẦNG CƠ SỞ

Cây / SVM / Logistic...
Huấn luyện theo K fold.
Mỗi mẫu chỉ lấy dự đoán
từ mô hình chưa thấy nó.

ĐẶC TRƯNG OOF

Một cột / mô hình cơ sở.
Các dự đoán ngoài fold
tạo đầu vào cho
mô hình tầng hai.

META-MODEL

Học cách phối hợp
các mô hình cơ sở.
Test chỉ dùng để
đánh giá cuối cùng.

Tránh rò rỉ: không huấn luyện tầng hai bằng dự đoán in-sample của tầng cơ sở.

Bài toán: phân loại mẫu khối u vú

569
quan sát

30
đặc trưng số

212
ác tính (nhãn 0)

357
lành tính (nhãn 1)

ĐẦU VÀO

Đặc trưng nhân tế bào: radius, texture, perimeter, area...

ĐẦU RA

Dự đoán ác tính hoặc lành tính cho một quan sát mới.

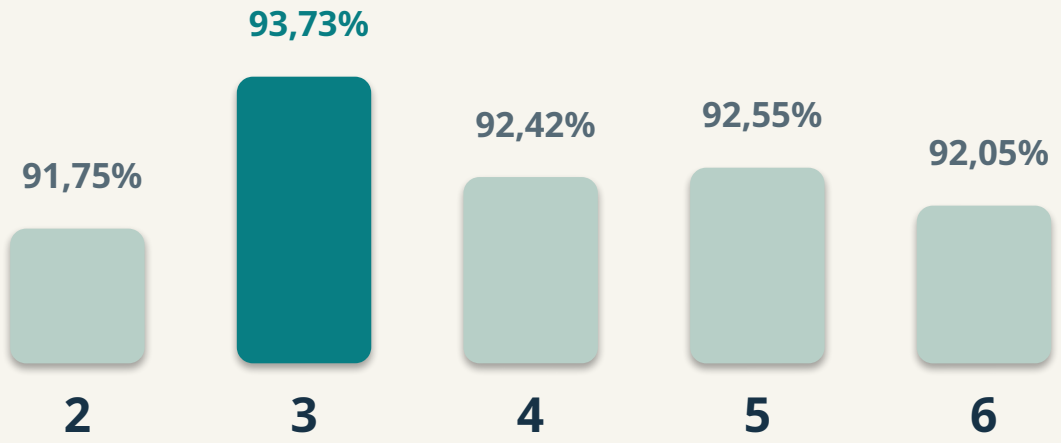
Mục đích học thuật; chưa xác thực lâm sàng. Dữ liệu có sẵn trong scikit-learn.

Thiết kế thí nghiệm giữ test ngoài quá trình chọn độ sâu

TRAIN 75% • 426 mẫu (159 ác / 267 lành)

TEST • 143 mẫu

CV phân tầng 5 fold trên train



Trục độ sâu 2–6; cột hiển thị phần vượt 90% CV.

CHỌN max_depth = 3

Balanced accuracy CV = 93,73%.
Chọn bằng cây đơn, dùng chung cho cả ba mô hình.
Không tối ưu riêng từng ensemble.

Nguồn: outputs/results.json • seed = 42

Tự viết hai phần cốt lõi: bootstrap và voting

01 LẤY MẪU & HUẤN LUYỆN

```
indices = rng.integers(
    0, n_samples, size=n_samples
)
tree = DecisionTreeClassifier(
    max_depth=self.max_depth,
    random_state=...
)
tree.fit(X[indices], encoded[indices])
self.estimators_.append(tree)
```

02 ĐẾM PHIẾU & CHỌN LỚP

```
counts = np.zeros((
    len(X), len(self.classes_)
), dtype=np.int64)
rows = np.arange(len(X))
for tree in self.estimators_:
    predicted = tree.predict(X).astype(int)
    np.add.at(counts, (rows, predicted), 1)

self.classes_[counts.argmax(axis=1)]
```

Cây cơ sở dùng thư viện; quy trình Bagging và hard voting do mã minh họa triển khai.

Bagging tự viết và Random Forest khác nhau ở đâu?

Thành phần	Manual Bagging	Random Forest (sklearn)
Lấy mẫu train	Bootstrap	Bootstrap
Đặc trưng khi chia	Xét toàn bộ 30 đặc trưng	Tập con ngẫu nhiên (√30 → 5)
Kết hợp cây	Đa số nhãn dự đoán	Trung bình xác suất lớp
Cấu hình seed 42	51 cây, max_depth = 3	51 cây, max_depth = 3

Ngẫu nhiên hóa đặc trưng giúp giảm tương quan; không bảo đảm mọi cây sẽ có lỗi khác nhau.

[2], [3] • RandomForestClassifier mặc định max_features='sqrt'

Bagging cải thiện nhẹ trên lần chia test này

Mô hình	Accuracy	Precision*	Recall*	F1*
Cây quyết định đơn	94,41%	94,12%	90,57%	92,31%
Bagging tự viết	**95,10%**	**94,23%**	**92,45%**	**93,33%**
Random Forest	95,10%	94,23%	92,45%	93,33%

* Lớp dương: ác tính = 0. Precision, Recall và F1 đều tính cho lớp này.

Accuracy tăng 0,70 điểm phần trăm; không suy ra ưu thế tổng quát từ một lần chia.

Seed 42 • test n = 143 • Nguồn: results.json • số liệu đã làm tròn

Đánh giá lại trên 10 lần chia phân tầng

Mỗi seed: chia lại train/test → chọn depth trên train → đánh giá ba mô hình.

Mô hình	Accuracy (%)	Recall ác (%)	F1 ác (%)
Cây quyết định đơn	92,80 ± 1,84	90,38 ± 4,66	90,30 ± 2,42
Bagging tự viết	**94,76 ± 1,59**	**92,45 ± 3,77**	**92,89 ± 2,13**
Random Forest	95,03 ± 1,85	92,26 ± 3,26	93,26 ± 2,39

Các tập test có thể giao nhau. SD mô tả độ biến thiên giữa các split, không phải khoảng tin cậy của các mẫu kiểm định độc lập.

Nguồn: repeated_evaluation.summary • seeds 42–51 • Mean ± SD mẫu

Out-of-bag: chỉ bỏ phiếu cho mẫu cây chưa học

94,13%

ACCURACY OOB

401 / 426 mẫu có dự đoán đúng

100%

COVERAGE

426 / 426 mẫu có ít nhất 1 phiếu

8–34

CÂY OOB / MẪU

Số phiếu mỗi mẫu có thể khác nhau

Mẫu nằm trong bootstrap của cây → Không nhận phiếu OOB từ cây đó.
Mẫu không được chọn vào bootstrap → Nhận 1 phiếu từ cây đó.

Depth đã chọn bằng CV trên train. OOB sau lựa chọn này không phải điểm kiểm định độc lập; test vẫn được giữ riêng cho đánh giá cuối ở mỗi seed.

Nguồn: results.json → oob • Manual Bagging, seed 42

Nhìn vào ca bỏ sót, không chỉ accuracy

CÂY ĐƠN

	Dự đoán ác	Dự đoán lành
Thật ác	48	5
Thật lành	3	87

BAGGING

	Dự đoán ác	Dự đoán lành
Thật ác	49	4
Thật lành	3	87

Bỏ sót ác tính: 5/53 → 4/53 | Recall: 90,57% → 92,45%

Bagging vẫn bỏ sót 4 ca; chưa đủ cơ sở sử dụng cho quyết định lâm sàng.

Seed 42 • Hàng = thật • Cột = dự đoán • Lớp dương = ác tính (0)

Phiếu bất đồng có thể dẫn đến dự đoán sai

Vị trí test 2 • Hàng dữ liệu gốc 291 • Nhãn thật: lành tính

Mẫu bất đồng đầu tiên theo thứ tự test

27 cây chọn ác tính
24 cây chọn lành tính

45 / 143 mẫu test có phiếu chia hai phía.

TỔNG 51 CÂY

27 : 24

Ác tính : Lành tính

Đa số chọn ác tính → Sai: nhãn thật là lành tính. Tỷ lệ phiếu không phải xác suất bệnh.

Nguồn: outputs/results.json → disagreement_example • Seed 42

Kết quả có ích, nhưng phạm vi kết luận còn hẹp

ĐÃ KIỂM CHỨNG

Bootstrap và hard voting.
10 seed, mean và SD mẫu.
OOB kèm coverage.
Test riêng ở mỗi seed.
Ví dụ bất đồng và lỗi.

GIỚI HẠN

Các holdout giao nhau.
Chọn depth bằng cây đơn.
OOB sau chọn depth.
Chưa kiểm định trên
quần thể độc lập.

PHÁT TRIỂN

Tuning riêng bằng CV.
Kiểm định dữ liệu ngoài.
So sánh AdaBoost, GBM.
Đánh giá đánh đổi lỗi.
Khảo sát số cây bằng CV.

Demo: hiểu cơ chế trước, xem kết quả sau

```
python examples/toy_bagging.py --output-dir outputs/toy
python de_5_ensemble_learning.py --repeats 1 --output-dir outputs/quick
```

Kết quả đầy đủ

results.json, report.md
Cấu hình, OOB và diễn giải.

Độ ổn định

summary_metrics.csv
stability.png: mean và SD.

Phân tích lỗi

confusion_matrices.png
voting_example.png: phiếu.

Cài dependencies theo README trước khi chạy. Không cần tải dữ liệu hoặc khóa API.

Ba điều cần nhớ

1

Đa dạng trước, tổng hợp sau

Nhiều mô hình giống nhau không tự tạo ra một tổ hợp tốt.

2

Bagging = Bootstrap + Aggregating

Mẫu có hoàn lại tạo các cây khác nhau; voting kết hợp dự đoán.

3

Đọc lỗi cụ thể, giữ kết luận có giới hạn

Bagging đạt 95,10% accuracy và bỏ sót 4/53 ca ác tính trên test này.

Tài liệu tham khảo

[1] Breiman (1996). Bagging Predictors. Machine Learning 24, 123–140.
https://doi.org/10.1007/BF00058655

[2] Breiman (2001). Random Forests. Machine Learning 45, 5–32.
https://doi.org/10.1023/A:1010933404324

[3] scikit-learn — Ensemble methods (Voting, Bagging, RF, Boosting, Stacking).
https://scikit-learn.org/stable/modules/ensemble.html

[4] scikit-learn — Breast Cancer Wisconsin Diagnostic dataset.
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

[5] scikit-learn — Decision Trees (Gini impurity, splitting criteria).
https://scikit-learn.org/stable/modules/tree.html

[6] Freund & Schapire (1997). A Decision-Theoretic Generalization...
https://doi.org/10.1006/jcss.1997.1504

[7] Wolpert (1992). Stacked Generalization. Neural Networks 5(2), 241–259.
https://doi.org/10.1016/S0893-6080(05)80023-1

Số liệu: outputs/results.json • Seeds 42–51 (tham chiếu 42) • NumPy 2.3.5 / scikit-learn 1.8.0

Câu hỏi thường gặp khi bảo vệ

Vì sao 51 cây?	Minh họa tổ hợp đủ lớn, tránh hòa ở hai lớp; chưa tối ưu số cây.
Vì sao không chuẩn hóa?	Các cây dùng phép chia theo ngưỡng, không dựa vào khoảng cách Euclid.
Test có bị dùng để tuning?	Không. Độ sâu chọn bằng CV trên train; test chỉ đánh giá cuối.
Bagging có luôn tốt hơn?	Không. Lỗi tương quan cao và mô hình cơ sở yếu có thể làm lợi ích thấp.
Vì sao nhấn mạnh recall?	Recall ác tính phản ánh tỷ lệ phát hiện và số ca ác tính bị bỏ sót.

Cách tính chỉ số với ác tính là lớp dương

Chỉ số	Công thức	Thay số Bagging
Accuracy	(TP + TN) / N	(49 + 87) / 143 = 95,10%
Precision ác tính	TP / (TP + FP)	49 / (49 + 3) = 94,23%
Recall ác tính	TP / (TP + FN)	49 / (49 + 4) = 92,45%
F1 ác tính	2TP / (2TP + FP + FN)	98 / (98 + 3 + 4) = 93,33%

Balanced accuracy dùng trong CV = (Recall ác tính + Recall lành tính) / 2.